

gcp dataflow intro

Dataflow

Course: Serverless Data Processing with Dataflow: Operations: https://www.cloudskillsboost.google/course_templates/264

Refer to this article <https://medium.com/@sasidharan01/streaming-data-with-dataflow-understanding-watermarks-custom-triggers-and-data-accumulation-f4275fe4c883>

<https://cloud.google.com/dataflow/docs/horizontal-autoscaling>

You can refer to [this](#) for docs.

"At a high level, Google Cloud Dataflow is a fully managed, serverless service for processing large-scale data. Its biggest strength is that it handles both batch and streaming data seamlessly within the exact same engine.

Under the hood, it uses the open-source Apache Beam SDK. This is a huge advantage because it gives us portability. I can write my data pipelines in Python or Java, and the logic isn't strictly locked into Google Cloud.

When it comes to execution, Dataflow completely removes the operational overhead. You just submit your job, and it automatically provisions the worker VMs and handles autoscaling. If we get a spike in streaming traffic, it scales up and down automatically. For streaming specifically, its Streaming Engine actually separates compute from storage, which keeps data latency really low.

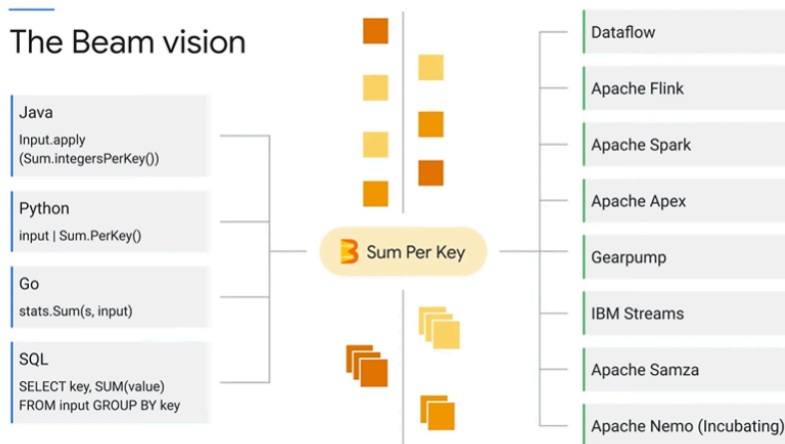
Conceptually, building a pipeline is pretty straightforward: you read data from a source into a distributed collection called a PCollection. Then you apply transformations. Since PCollections are immutable, every transform creates a new PCollection, and finally, you write the output to a sink like BigQuery.

Overall, I'd choose Dataflow for heavy ETL, real-time stream analytics, or prepping data for machine learning—basically, anytime I need to focus on the data logic rather than managing the underlying infrastructure."

- Cloud version for [Apache Beam](#).
- Unified API for Batch and Streaming data and run same code on execution frameworks like Spark, Flink, Apex, Cloud dataflow, etc.
- Preferred for new Hadoop or Spark infrastructure development
- Process batch or stream data.
- Serverless, fast, scalable, fault-tolerant.
- Multi-Step processing data. Eg: Wordcount.
- Cloud dataflow executed as jobs where one or more worker carry out specific tasks
- Supports SQL, Java, Python
- When you run a job on Cloud Dataflow, it spins up a cluster of virtual machines, distributes the tasks in your job to the VMs, and dynamically scales the cluster based on how the job is performing.
- Stackdriver integration for logging and monitoring
- Used for data processing, ELT(Extract Transform Load), filter, group for data sets.
- Can read data from multiple sources, can kick off multiple cloud functions in parallel and can also writer to multiple sinks (like BigQuery, BigTable etc.)
- When you update a job on the Dataflow service, you replace the existing job with a new job that runs your updated pipeline code
- Jobs can be created with inbuilt templates, or notebook instances(write jobs in Java/Python/SQL)
- The Dataflow SDK provides a transform component. It is responsible for the data processing operation. You can use conditional, for loops, and other complex programming structure to create a branching pipeline.
- Can apply [windowing](#) to streams for rolling average for the window, max in a window etc.
- You can stop a Dataflow job in the following two ways:
 - Canceling a job. This method applies to both streaming and batch pipelines. Canceling a job stops the Dataflow service from processing any data, including buffered data
 - Draining a job. This method applies only to streaming pipelines. Draining a job enables the Dataflow service to finish processing the buffered data while simultaneously ceasing the ingestion of new data.
- Updating the pipeline:
 - If any major change to windowing transformation (like completely changing window fn from fixed to sliding) in Beam/Dataflow/you want to stop pipeline but want inflight data --> use Drain option.
 - For all other use cases and Minor changing to windowing fn (like just changing window time of sliding window) --> Use Update with Json mapping.
- Important IAM roles -
 - **dataflow.developer** role enable the developer interacting with the Cloud Dataflow job , with data privacy.

- **dataflow.worker** role provides the permissions necessary for a *Compute Engine service account* to execute work units for a Dataflow pipeline

The Beam vision



[A Simple Dataflow Pipeline \(Python\) 2.5 | Google Cloud Skills Boost](#)

[Use Flexible Resource Scheduling in Cloud Dataflow | Google Cloud](#)

[Use Dataflow Shuffle for batch jobs | Google Cloud](#)

Common Concepts in Dataflow

- **Pipeline:** encapsulates series of computations that accepts input data from external sources, transforms data to provide some useful intelligence, and produce output
- **PCollections:** abstraction that represents a potentially distributed, multi-element data set, that acts as the pipeline's data. PCollection objects represent input, intermediate, and output data. The edges of the pipeline.
- **Transforms:** operations in pipeline. A transform takes a PCollection(s) as input, performs an operation that you specify on each element in that collection, and produces a new output PCollection. Composite transforms are multiple transforms: combining, mapping, shuffling, reducing, or statistical analysis.
- **Pipeline I/O:** the source/sink, where the data flows in and out. Supports read and write transforms for a number of common data storage types, as well as custom.
- **Windowing:** Windowing a PCollection divides the elements into windows based on the associated event time for each element.
- **Triggers:** Allows specifying a trigger to control when (in processing time) results for the given window can be produced. Triggers determines when a Window's contents should be output based on certain criteria being met. Types of triggers are :
 - Time based triggers
 - Data Driven triggers
 - Composite triggers
- **Watermark:** It is a threshold that indicates when Dataflow expects all of the data in a window to have arrived. If new data arrives with a timestamp that's in the window but older than the watermark, the data is considered **late data**.
 - Watermark is a heuristic that tracks how far behind the system is in processing data from the event time. Where in event time does processing occur?
- **ParDo:** It is a parallel processing function which can transform elements of an input PCollection to an output PCollection.
 - ParDo is a common intermediate step in a pipeline
 - You might use it to extract certain fields from a set of raw input records or convert raw input into a different format.
 - You might also use ParDo to convert process data into an output format, like table rows for BigQuery or strings for printing.
 - You can use ParDo to consider each element in a PCollection and either output that element to a new collection or discard it.
 - If your input PCollection contains elements that are of a different type or format than you want, you can use ParDo to perform a conversion on each element and output the result to a new PCollection.
-
- **DoFn:** It is a template which is used to create user defined functions that are referenced by ParDo. A DoFn is a Beam SDK class that defines a distributed processing function.